

Sahoulat API Documentation

REST API for the Sahoulat mobile app (customers + professionals). Built on Laravel 12 + Sanctum. All routes are prefixed with `/api` , e.g. `https://your-domain.test/api/login` .

Tested end-to-end against the full booking → escrow payment → completion → wallet payout lifecycle, jobs/bids, emergencies, contracts, and disputes.

1. Conventions

Headers

Every request should send:

```
Accept: application/json
```

Authenticated requests must also send:

```
Authorization: Bearer <token>
```

Requests with a JSON body should send:

```
Content-Type: application/json
```

File upload endpoints (photos, KYC documents) use `multipart/form-data` instead.

Auth model

Token-based via **Laravel Sanctum**. There is no session/CSRF — every device that logs in gets its own long-lived bearer token (`device_name` you send becomes the token's label, so a user can be logged in on multiple devices and revoke them individually).

Roles

A user has exactly one `role` : `consumer` , `provider` , or `job_seeker` (`job_seeker/admin` flows are not covered by this API — it's scoped to the customer + professional apps). Role is fixed at registration and determines which route group (`/api/consumer/*` or `/api/provider/*`) the account can call. Cross-role calls return `403` .

Standard error shapes

401 — missing/invalid/expired token:

```
{ "message": "Unauthenticated." }
```

403 — wrong role, or a policy denied the action:

```
{ "message": "This account type cannot access this resource." }
```

404 — model not found, or (deliberately, for security) hidden because it doesn't belong to you / isn't active.

422 — validation errors (standard Laravel shape):

```
{
  "message": "The email field is required.",
  "errors": { "email": ["The email field is required."] }
}
```

```
}
```

422 is also used for business-rule rejections (e.g. "That time slot is no longer available"), in the simpler `{ "message": "..."}` shape.

409 — race condition (e.g. two providers accepting the same emergency request at once).

Pagination

List endpoints that paginate return:

```
{
  "...collection key...": [ /* items */ ],
  "pagination": { "current_page": 1, "last_page": 3, "total": 42 }
}
```

Money

All amounts are numbers (PKR), not strings, e.g. `"price": 3000` not `"price": "3000.00"`.

2. Auth

POST `/api/register`

Auth: none **Body:** | Field | Type | Notes | |---|---|---| | `name` | string | required | | `email` | string | required, unique | | `phone` | string | required — any format accepted; normalized server-side to `03XX-XXXXXXX` (see note below) | | `role` | string | required — `consumer`, `provider`, or `job_seeker` | | `password` | string | required, min 8 | | `password_confirmation` | string | required, must match `password` | | `referral_code` | string | optional | | `device_name` | string | required — label for this token, e.g. `"iPhone 15"` |

Pakistani phone numbers are reformatted to `03XX-XXXXXXX` on save (a leading `+92 / 92` country code is stripped and replaced with a trunk `0` first). The stored/returned value may therefore differ from what was submitted — e.g. `+923001234567` or `03001234567` both come back as `0300-1234567`. Numbers that aren't 11 digits after stripping non-digits are left as-is.

Response 201 :

```
{
  "user": {
    "id": 22, "name": "Test Consumer", "email": "test@example.com",
    "phone": "0300-1234567", "role": "consumer", "referral_code": "0G1EGG",
    "credit_balance": 0, "is_suspended": false,
    "provider_status": null,
    "created_at": "2026-07-21T12:12:07+00:00"
  },
  "token": "1|dFx1zYRMLAFDEVy2Ba1JMLVvxzDkRPhobybCfHbN4541fa29"
}
```

`provider_status` is only meaningful when `role` is `provider` (mirrors `provider_profiles.status`: `draft` → `pending` → `approved` / `rejected`).

POST `/api/login`

Auth: none **Body:** `email`, `password`, `device_name` (all required) **Response 200 :** same shape as register. **Errors:** **422** if credentials don't match, or if the account is suspended.

POST `/api/forgot-password`

Auth: none. Rate limited (5/min per email+IP). **Body:** `email` (required) **Response:** `{ "message": "We sent a 6-digit code to your email." }` Emails a 6-digit OTP (15-minute expiry) via the same `password_reset_tokens` table and mailable the web app's forgot-password page uses. **Errors:** **422** if no user has that email.

POST /api/reset-password

Auth: none. Rate limited (5/min per email+IP on the route, plus 5 wrong-OTP attempts per email+IP before a 5-minute logout).

Body: email, otp (6 digits, from the emailed code), password (required, confirmed, min 8) **Response:** { "message": "Your password has been reset — please log in." } Verifies the OTP, sets the new password, and revokes the reset token (not the user's Sanctum tokens — existing logged-in devices stay logged in). **Errors:** 422 if the OTP is invalid/expired/rate-limited, or the email doesn't match a user.

POST /api/logout

Auth: required — revokes only the token used for this request (this device). **Response:** { "message": "Logged out." }

POST /api/logout-all

Auth: required — revokes every token for the user (all devices). **Response:** { "message": "Logged out of all devices." }

GET /api/me

Auth: required **Response:** { "user": { ...same shape as register... } }

PUT /api/profile

Auth: required **Body:** name (required), email (required, unique except self), phone (optional — normalized to 03XX-XXXXXXX the same way as registration) **Response:** { "user": { ... } }

PUT /api/profile/password

Auth: required **Body:** current_password (required, must match), password (required, confirmed, min 8), password_confirmation **Response:** { "message": "Password updated." }

DELETE /api/profile

Auth: required **Body:** password (required, must match current password) **Response:** { "message": "Account deleted." }

Anonymizes the user's name/email/phone/avatar, sets a random unusable password, deactivates the account (suspended_at), and revokes all Sanctum tokens. Booking/payment history is preserved (no hard delete) to avoid breaking referential integrity.

3. Shared / public browsing

None of these require auth unless noted.

GET /api/categories

Active categories with their active services (the "browse services" catalog, cached 1h server-side). **Response:**

```
{
  "categories": [
    {
      "id": 1, "name": "AC Repair & Service", "slug": "ac-repair-service",
      "description": "...", "icon": "ac", "image_url": null,
      "services": [
        { "id": 2, "category_id": 1, "name": "AC Gas Refill", "slug": "ac-gas-refill",
          "description": "...", "base_price": 3500, "visit_charge": null,
          "duration_minutes": 90, "is_active": true }
      ]
    }
  ]
}
```

icon is either a legacy icon-name key (e.g. "ac", rendered client-side from a bundled icon set — true for any category that predates icon uploads) or, once a category has an uploaded icon image, the ready-to-use absolute URL for it (e.g.

"https://sahoulat.com/storage/categories/xyz.jpg"). Check whether it starts with http to tell which case you're in. Same behavior on the nested category.icon in ServiceResource.

`visit_charge` (nullable) is a fixed, non-negotiable fee some services carry, payable if a provider cancels after inspecting the job in person — separate from and in addition to the service's `base_price`. **Disclose it up front, before booking** — the web app shows it as a prominent callout on the service detail page and a small line on service cards, precisely because it can surprise a customer if they only learn about it after the fact. See `POST /provider/bookings/{id}/status` (`visit_charge_method` / `visit_charge_screenshot`) for how it actually gets collected, and the `visit_charge` object on `BookingResource` for the collected record.

`GET /api/services`

All active services (flat list, e.g. for search/typeahead). Response: `{ "services": [ { ...ServiceResource... } ] }`

`GET /api/services/{slug}`

Service detail + approved providers offering it (cheapest first, paginated) + 3 related services. "Approved" here excludes providers currently suspended for unpaid cash-commission debt — they're temporarily not eligible for new bookings until they settle up.

Response:

```
{
  "service": { "id": 2, "name": "AC Gas Refill", "...": "..." },
  "providers": [
    { "provider_profile_id": 21, "provider": { "...ProviderProfileResource..." }, "price": 3000 }
  ],
  "providers_pagination": { "current_page": 1, "last_page": 1, "total": 1 },
  "related_services": [ { ... } ]
}
```

`GET /api/providers`

Public provider directory — approved and not currently suspended for unpaid cash-commission debt. **Query:** `q` (search name/bio/city/service), `city` (exact match). Both optional. **Response:**

```
{
  "providers": [ { ...ProviderProfileResource... } ],
  "pagination": { "current_page": 1, "last_page": 1, "total": 5 },
  "cities": ["Islamabad", "Karachi", "Lahore"]
}
```

`GET /api/providers/{id}`

One provider's public profile: bio, services offered (with prices), portfolio photos, and latest 10 reviews. **Response:** `{ "provider": { ... }, "reviews": [ { "id":1, "rating":5, "comment":"...", "consumer_name":"...", "service_name":"...", "created_at":"..." } ] }`

`GET /api/providers/{id}/services/{serviceId}/availability`

Bookable dates + time slots for a specific provider/service pairing — call this before showing the booking picker. **Query:** `date` (optional, `YYYY-MM-DD` ; defaults to the first bookable date). **Response:**

```
{
  "price": 3000, "duration_minutes": 90,
  "dates": [
    { "value": "2026-07-21", "label": "Today — 21 Jul" },
    { "value": "2026-07-22", "label": "Wed, 22 Jul" }
  ],
  "selected_date": "2026-07-21",
  "slots": [
    { "value": "09:00", "label": "9:00 AM", "available": false },
    { "value": "10:00", "label": "10:00 AM", "available": true }
  ]
}
```

`GET /api/subscription-plans`

Active maintenance/AMC plans. Response: `{ "plans": [ { "id":1, "name":"...", "slug":"...", "service": { ... }, "frequency_months":1, "frequency_label":"Monthly", "total_visits":12, "price_per_visit":2000, "is_active":true } ] }`

`GET /api/cities`

Cities with at least one approved provider — for populating city pickers, purely informational (this list has no bearing on geo-fencing, which is boundary-only now, not city-based). Response: `{ "cities": ["Islamabad", "Karachi", "Lahore"] }`

4. Shared / authenticated (any role)

All require Authorization: Bearer <token> .

GET /api/notifications

Paginated, newest first. Response: `{ "notifications": [ {...} ], "unread_count": 3, "pagination": {...} }` Each notification: `{ "id", "type", "title", "body", "url", "is_read", "read_at", "created_at" }`

POST /api/notifications/{id}/read

Marks one notification read. Response: `{ "notification": {...} }`

POST /api/notifications/read-all

Response: `{ "message": "All notifications marked as read." }`

GET /api/bookings/{id}/room

Chat history + tracking pin(s) for a booking (both consumer and provider on the booking can call this). **Response:**

```
{
  "is_communicable": true,
  "can_share_location": true,
  "messages": [ { "id":1, "sender_id":22, "sender_name":"Test Consumer", "body":"...", "created_at":"..." } ],
  "latest_tracking": { "id":1, "latitude":24.86, "longitude":67.01, "note":null, "created_at":"..." },
  "tracking_history": [ { "id":1, "latitude":24.86, "longitude":67.01, "note":null, "created_at":"..." }, "...up to the most recent" ],
  "destination": { "latitude":24.90, "longitude":67.05, "address":"House 1, Karachi" }
}
```

`destination` is the booking's own address coordinates (`null` if the booking has none) — feed `latest_tracking` as the origin and `destination` as the destination into the Google Directions API client-side to draw the actual driving route + ETA, the same way the web app's booking room does (`google.maps.DirectionsService / DirectionsRenderer`). It's static for the life of the booking, so fetch it once alongside everything else here rather than re-deriving it.

`can_share_location` is `true` only while the booking is `confirmed` — once it flips to `in_progress` the provider has arrived, so there's nothing left to route to and this goes back to `false` . Stop sending tracking pings (and hide the "share my location" control) the moment this flips, rather than only reacting to a `422` from the tracking endpoint.

POST /api/bookings/{id}/messages

Body: `body` (string, required, max 2000) **Response 201 :** `{ "message_data": {...} }` — broadcast in realtime over the existing Reverb/Echo channel for the booking.

POST /api/bookings/{id}/tracking

Post a live location pin (typically the provider en route). **Body:** `latitude` , `longitude` (required), `note` (optional). **Response 201 :** `{ "tracking": {...} }`

POST /api/bookings/{id}/dispute

Body: `reason` (string, required, max 2000), `photos[]` (optional, up to 5 images, 8MB each, multipart) — evidence for the complaint. **Response 201 :** `{ "dispute": {...} }`

GET /api/disputes/{id}

Response: `{ "dispute": { "id", "reference", "opened_by_role", "reason", "status", "resolution", "resolution_note", "resolved_at", "created_at", "photos":[{"id":1, "url":...} ] }`

5. Consumer API (/api/consumer/* , role `consumer` required)

GET /api/consumer/dashboard

Home-screen summary. Response: { "recent_bookings": [{...BookingResource, up to 5...}] }

Addresses

| Method | Path | Body | Notes | |---|---|---| | GET | /addresses | - | { "addresses": [...] } | | POST | /addresses | label , address , city (required); latitude , longitude , is_default (optional) | 201 , { "address": {...} } | | PUT | /addresses/{id} | same as POST | { "address": {...} } | | DELETE | /addresses/{id} | - | { "message": "Address removed." } |

Address shape: { "id","label","address","city","latitude","longitude","is_default" }

Geo-fencing on POST/PUT: same boundary check as bookings/emergencies/etc. (see POST /bookings above) — when geo-fencing is on, latitude / longitude are checked against active ServiceArea boundaries before the address is saved. Outside every drawn boundary → 422 with { "message": "Sorry, we don't operate in the \"{address}\" area yet." } .

Bookings (direct booking flow)

GET /bookings — paginated, newest first. { "bookings": [...], "pagination": {...} }

POST /bookings — create a direct booking with a specific approved provider. | Field | Required | Notes | |---|---|---| | provider_profile_id | yes | must be an approved provider | | service_id | yes | must be a service that provider actively offers | | scheduled_date | yes | one of the values from the availability endpoint | | scheduled_time | yes | HH:MM , must be an available slot | | address | yes | | | latitude , longitude | no | | | notes | no | max 1000 |

Response 201 : { "booking": {...BookingResource...} } . 422 if the *customer's own address* (not the provider's city, as it used to be — see below) is outside our served areas while geo-fencing is on, or if the slot just got taken.

Geo-fencing (applies here and to POST /emergencies , POST /job-posts , POST /contracts , POST /subscriptions — all identical behavior): when the admin-side "Geo-fencing" setting is on, each of these is checked against active ServiceArea rows before it's created. This is purely a point-in-polygon check against **real coordinates** — every service area has an admin-drawn boundary (a city field/string match is no longer part of this at all), so you must supply real latitude / longitude , not just an address string, or the request is rejected outright. A location outside every drawn boundary is always treated as outside the service area, with no city-name fallback. If it fails, the response is 422 with a message explaining the location isn't served. When geo-fencing is off (default) or no service areas are configured yet, nothing is blocked.

GET /bookings/{id} — { "booking": {...} } (full detail incl. payments , review , dispute)

POST /bookings/{id}/cancel — only while cancellable by the consumer (not yet in progress). Auto-refunds escrow if already paid. { "message": "...", "booking": {...} }

GET /bookings/{id}/payment-options — { "gateways": [{ "key": "mock", "label": "Test payment (sandbox)" }, { "key": "cash", "label": "Cash" }, { "key": "bank_transfer", "label": "Bank transfer" }], "max_credit_applicable": 0, "amount": 3000, "company_account": {...} } . mock only ever appears outside production (APP_ENV) or when explicitly forced on via

MOCK_PAYMENTS_ENABLED — it must never be reachable on the live site, since it always succeeds instantly with no real money.

jazzcash / easypaisa only appear once their env credentials + *_ENABLED=true are configured. **cash and bank_transfer are always included** (same reasoning as milestones below) so pre-payment never dead-ends into an empty gateway list when no online gateway is configured. company_account is the same shape as completion-payment-options below — needed for the bank-transfer instructions.

POST /bookings/{id}/pay — Body: gateway (required unless referral credit fully covers the amount — must be one of the keys returned by payment-options above), apply_credit (bool, optional — applies available referral credit first), screenshot (required if gateway=bank_transfer , image, multipart, max 8MB). Response 201 : { "message": "...", "payment": {...} } , or, for gateways needing an off-site redirect: { "status": "pending", "redirect_url": "...", "redirect_fields": {...} } .

`gateway=cash` here is a *commitment*, not an instant charge — nothing can really be "prepaid" in cash before the provider has shown up. The payment is recorded `pending` and only actually settled (commission deducted, invoice emailed) once the job is later marked complete. `gateway=bank_transfer` behaves like a real gateway: it's recorded `pending` with the screenshot and, once an admin verifies it, held in escrow (not released) until the job is completed and the consumer calls `/release` — unlike the completion-payment bank transfer below, which verifies straight to released since the job is already done by then.

POST /bookings/{id}/release — release escrow to the provider once completed & undisputed. { "message": "...", "booking": {...} }

POST /bookings/{id}/review — Body: `rating` (1-5, required), `comment` (optional, max 1000). Only once, only after completion. Response 201: { "review": {...} }

GET /bookings/{id}/completion-payment-options — for bookings that skip pre-payment and only ask for money once the job is actually done (see `permissions.needs_completion_payment` below, and `Booking::needsCompletionPayment()`). 404 if this booking isn't one of those. Response:

```
{
  "amount": 1000,
  "methods": ["cash", "bank_transfer"],
  "company_account": {
    "bank_name": "Bank Al Habib Ltd",
    "account_title": "Sahoulat Facility Management Services",
    "account_number": "5052-0081-001208-01-4",
    "iban": "PK15BAHL50520081001208014",
    "swift_code": "BAHLPKKA",
    "branch_name": "Islamic Midway Commercial \"A\" Bahria Town Karachi",
    "branch_code": "5052",
    "branch_address": "Showroom # 1, SQ Trade Center, Plot A-118 Midway Commercial \"A\" Bahria Town Karachi Pakistan",
    "jazzcash_number": "",
    "easypaisa_number": ""
  }
}
```

This is Sahoulat's own receiving account for a manual bank transfer — display it in full so the customer can copy every field (`iban` / `swift_code` matter for interbank/international transfers). `jazzcash_number` / `easypaisa_number` are currently empty (not yet configured) — omit those rows client-side when blank, same as the web app does.

POST /bookings/{id}/completion-payment — record how a just-completed job got paid. Body: `method` (required, `cash` | `bank_transfer`), `screenshot` (required if `method` is `bank_transfer`, image, multipart, max 8MB). `cash` is recorded immediately (commission is deducted from the provider's wallet automatically) and the customer is emailed a PDF invoice right away. `bank_transfer` needs an admin to verify the screenshot first (web-only step) — the invoice email goes out once that happens. Response 201: { "message": "...", "payment": {...} }. 422 if this booking isn't awaiting completion payment.

Booking resource shape (used throughout)

```
{
  "id": 1, "reference": "BK-Z6POMA", "status": "confirmed",
  "service": { "...ServiceResource..." },
  "provider": { "...ProviderProfileResource..." },
  "consumer": { "id": 22, "name": "Test Consumer", "phone": "0300-1234567" },
  "scheduled_date": "2026-07-22", "scheduled_time": "10:00",
  "price": 3000, "duration_minutes": 90, "address": "House 1, Karachi",
  "latitude": null, "longitude": null, "notes": null,
  "cancelled_by": null, "cancellation_reason": null,
  "completion_notes": null,
  "confirmed_at": "...", "started_at": null, "completed_at": null, "cancelled_at": null,
  "created_at": "...",
  "before_photos": [ { "id": 1, "url": "https://..." } ],
  "after_photos": [ { "id": 2, "url": "https://..." } ],
  "visit_charge": null,
  "payments": [ {...} ], "review": null, "dispute": null,
  "permissions": {
    "can_cancel": true, "is_payable": true, "needs_completion_payment": false, "is_reviewable": false,
    "is_disputable": false, "is_communicable": true, "can_share_location": false,
    "is_provider": false
  }
}
```

`permissions` tells the app which action buttons to show — always trust this over re-deriving the logic client-side. `before_photos` / `after_photos` are only present when the booking detail endpoint eager-loads them (both consumer and provider `GET /bookings/{id}` do). `visit_charge` is non-null only when a provider cancelled an in-progress booking after collecting the visit charge on inspection (see the provider status endpoint below) — shape: `{ "amount": 500, "method": "cash", "screenshot_url": null, "collected_at": "..."}` . This money is never part of commission/wallet accounting — it's the provider's directly.

Jobs (post & bid flow)

`GET /jobs` — `{ "jobs": [...], "pagination": {...} }`

`POST /jobs` — post a job for open bidding. Multipart if attaching photos. | Field | Required | |---| | `service_id` | yes | | `description` | yes, max 2000 | | `budget` | no, numeric | | `preferred_date` | no, date ≥ today | | `address`, `city` | yes | | `latitude`, `longitude` | no | | `photos[]` | no, up to 5 images, 5MB each |

Response 201: `{ "job": {...JobPostResource...} }`

`GET /jobs/{id}` — full detail with all bids (each bid includes the bidding provider's profile).

`POST /jobs/{id}/cancel` — only while `status = open`. Rejects any pending bids too.

`POST /jobs/{id}/bids/{bidId}/accept` — accepts a bid, creates a confirmed `Booking`, rejects all other pending bids on that job, notifies the winning + losing providers. `{ "message": "...", "booking": {...} }`. 422 if the slot clashes with another confirmed booking for that provider, or the proposed time has passed.

JobPost resource shape

```
{
  "id": 1, "reference": "JOB-PZYQ40", "status": "open",
  "service": {...}, "consumer": { "id": 22, "name": "..."},
  "description": "...", "budget": null, "preferred_date": null,
  "address": "...", "latitude": null, "longitude": null, "city": "Karachi",
  "photos": [ { "id": 1, "url": "https://.../job-photos/1/xyz.jpg" } ],
  "bids_count": 2, "pending_bids_count": 1,
  "bids": [ {...BidResource...} ],
  "my_bid": null,
  "awarded_at": null, "created_at": "..."
}
```

Contracts (multi-service projects, admin-quoted)

`GET /contracts` — `{ "contracts": [...], "pagination": {...} }`

`POST /contracts` — submit for a manual admin quote. Multipart if attaching photos. | Field | Required | |---| | `title`, `description` | yes | | `preferred_start_date` | no | | `address`, `city` | yes | | `latitude`, `longitude` | no | | `photos[]` | no, up to 8 images | | `items[]` | yes, min 1 — each: `service_id` (required), `quantity` (required, 1-100), `notes` (optional) |

Response 201: `{ "contract": {...} }` — status starts as `submitted`; an admin later sets `quoted_total` and per-milestone amounts.

`GET /contracts/{id}` — full detail incl. `items`, `photos`, `milestones`.

`POST /contracts/{id}/accept` / `POST /contracts/{id}/reject` — only while `status = quoted`.

`POST /contracts/{id}/cancel` — only while cancellable.

`GET /contracts/{id}/milestones/{milestoneId}/payment-options` — same shape as booking payment-options, **plus cash and bank transfer are always included** (unlike bookings, a milestone has no pay-after-completion fallback, so these two are the reliable path while no online gateway is configured). Also includes `company_account` (see the completion-payment shape above) for the bank-transfer instructions. Response:

```
{
  "gateways": [
    {"key": "mock", "label": "Test payment (sandbox)"},
    {"key": "cash", "label": "Cash"},
  ]
}
```

```
{
  "key": "bank_transfer", "label": "Bank transfer"
},
"max_credit_applicable": 0,
"amount": 3000,
"company_account": { "...": "same shape as the booking completion-payment endpoint" }
}
```

POST /contracts/{id}/milestones/{milestoneId}/pay — Body: `gateway` (required unless credit fully covers it — one of the keys from `payment-options`), `apply_credit` (bool, optional), `screenshot` (required if `gateway=bank_transfer`, image, multipart, max 8MB).

- `gateway=cash` — trusted immediately, no proof required (same as a real gateway success): the payment and milestone both move straight to escrow.
- `gateway=bank_transfer` — the payment stays `pending` until an admin verifies the screenshot in the admin panel; only then does it move to escrow. The consumer isn't blocked from anything in the meantime — this is just slower than cash.
- A real gateway key — unchanged, synchronous success moves straight to escrow (or `{"status":"pending","redirect_url":...}` for an off-site redirect gateway).

Response 201: `{ "message": "...", "payment": {...} }` in all cases (except the off-site redirect case, same shape as booking `/pay`).

Emergencies (SOS)

GET /emergencies — `{ "emergencies": [...], "pagination": {...} }`

POST /emergencies — broadcasts to nearby approved providers offering that service in that city (realtime + notification). | Field | Required | |---|---| | `service_id` | yes, must be active **and** flagged emergency-available — 422 otherwise | | `address`, `city` | yes | | `latitude`, `longitude` | no | | `notes` | no |

Send `latitude / longitude` when you have them (e.g. from the map picker) — they carry straight through to the `Booking` once a provider accepts, same as a direct booking's coordinates.

Response 201: `{ "emergency": {...} }`

GET /emergencies/{id} — includes `matched_provider` and `booking` once accepted, and `quoted_price` once an admin has quoted it.

POST /emergencies/{id}/cancel — while `open` or `quoted`.

POST /emergencies/{id}/accept-quote / POST /emergencies/{id}/decline-quote — respond to an admin's price quote (`status` must currently be `quoted`, i.e. `quoted_price` is set). Accepting moves it to `accepted` (admin assigns a provider next); declining moves it to `declined`, a terminal state. 422 if there's no pending quote. Response: `{ "message": "...", "emergency": {...} }`

Subscriptions (maintenance plans)

GET /subscriptions — `{ "subscriptions": [...], "pagination": {...} }`

POST /subscription-plans/{planSlug}/subscribe Body: `address`, `city` (required), `latitude`, `longitude` (optional), `start_date` (required, date ≥ today). Response 201: `{ "subscription": {...} }` — status starts `pending_assignment` until admin assigns a provider.

GET /subscriptions/{id} — includes `provider` and past `bookings` once active.

POST /subscriptions/{id}/cancel — Body: `reason` (optional). Already-scheduled visits are unaffected.

6. Provider API (/api/provider/* , role `provider` required)

Most endpoints beyond onboarding require an **approved** `ProviderProfile` — calling them before approval returns empty collections (list endpoints) or a 403 from the `actAsApprovedProvider` gate (action endpoints).

GET /api/provider/dashboard

Home-screen summary: counters, wallet balances, 6-month earnings trend, completion rate, average response time, bid win rate, today's schedule.

```
{
  "profile": {...ProviderProfileResource... or null},
  "pending_bookings": 0, "available_jobs": 2, "open_emergencies": 1,
  "wallet_available": 2700, "wallet_escrow": 0,
  "earnings_month": 2700, "earnings_total": 2700, "earnings_delta": 100,
  "earnings_series": [ {"label":"Feb","value":0}, "...", {"label":"Jul","value":2700} ],
  "jobs_completed": 1, "active_bookings": 1, "completion_rate": 100,
  "response_minutes": 0, "bid_win_rate": 100, "bids_pending": 0,
  "today_schedule": [ {...BookingResource...} ]
}
```

Onboarding / KYC

GET /onboarding — profile + uploaded documents + step progress + what's missing.

```
{
  "profile": {...},
  "documents": [ { "id":1, "type":"cnic_front", "original_name":"...", "download_url":"...", "created_at":"..." } ],
  "document_types": { "cnic_front": {"label":"CNIC – Front","required":true}, "...": "..." },
  "steps": [
    { "label": "Your details", "done": true },
    { "label": "KYC documents", "done": false, "uploaded": 1, "required": 3 },
    { "label": "Review", "done": false, "submitted": false }
  ],
  "missing": ["CNIC – Back", "Selfie holding CNIC"],
  "can_submit": false
}
```

PUT /onboarding — Body: experience_years , city , cnic_number (required); business_name , bio , address , latitude , longitude (optional). Blocked (422) once submitted/approved. cnic_number accepts any format but is normalized and stored as 42101-1234567-8 (13 digits required after stripping non-digits — an unexpected digit count is left as submitted).

POST /onboarding/documents — multipart. Body: type (one of the document_types keys), file (jpg/jpeg/png/pdf, max 4MB). Replaces any existing document of the same type. Response 201 : { "document": {...} }

GET /onboarding/documents/{id} — streams the private file (owner only). Not JSON — returns the raw file.

DELETE /onboarding/documents/{id} — { "message": "Document removed." }

POST /onboarding/submit — moves profile to pending for admin review once details + all required documents are present. 422 with a listing of what's missing otherwise.

Services offered

Method	Path	Body	Notes
GET	/services	-	{ "offered": [...], "available": [...], "booking_counts": {...} } — empty until approved
POST	/services	service_id (required, not already offered), price (required)	201 , { "provider_service": {...} }
PUT	/services/{id}	price (required), is_active (optional bool)	{ "provider_service": {...} }
DELETE	/services/{id}	-	{ "message": "..." }

Bookings

GET /bookings — **Query:** status (all | pending | confirmed | in_progress | completed | cancelled , default all), q (search reference/address/consumer/service name). Response: { "bookings": [...], "counts": {"all":5,"pending":1,...}, "filter": "all", "pagination": {...} }

GET /bookings/{id} — full detail.

POST /bookings/{id}/status — multipart when completing (photos) or cancelling with a bank-transfer visit charge (screenshot). Valid transitions: pending ?(confirm/decline)? , confirmed ?(start/cancel)? , in_progress ?(complete/cancel)? .

Declining/cancelling auto-refunds escrow.

| Body field | Required when | Notes | |---|---|---| | `action` | `always` | `confirm` | `decline` | `start` | `complete` | `cancel` | | `cancellation_reason` | `no` | max 1000, used for `decline` / `cancel` | | `completion_notes` | `no` | max 1000, used for `complete` | | `before_photos[]` | `no` | optional, 0–6 images, 5MB each — proof of work, recommended but not enforced | | `after_photos[]` | `no` | optional, 0–6 images, 5MB each | | `visit_charge_method` | `action=cancel` **and** the booking is currently `in_progress` | `cash` | `bank_transfer` — see below | | `visit_charge_screenshot` | same, **and** `visit_charge_method=bank_transfer` | image, max 8MB |

Two things worth calling out:

- `complete` **accepts optional before/after photos** and automatically generates an invoice for the booking, emailed to the customer once payment is confirmed.
- `cancel` **while in_progress** is a distinct scenario from cancelling a `confirmed` (not-yet-visited) booking: it means the provider inspected the job on-site and the customer decided not to proceed. In that case `visit_charge_method` is required — it records the provider's visit charge as collected (see `visit_charge` on the booking resource above) entirely outside commission/wallet accounting. Cancelling a `confirmed` booking (before any visit happened) does **not** need these fields.

Response: { "message": "...", "booking": {...} }

Jobs & bids

`GET /jobs` — open jobs matching services this provider offers; each job includes `my_bid` (null if not yet bid).

`GET /jobs/{id}` — { "job": {...}, "offers_service": true, "slot_options": [{"value":"09:00","label":"9:00 AM"}, ...] }

`POST /jobs/{id}/bids` — Body: `amount` (required, numeric), `proposed_date` (required, $\geq$ today), `proposed_time` (required, one of `slot_options`), `message` (optional). One bid per provider per job. Response 201: { "bid": {...} }

`GET /bids` — **Query:** `status` (`all` | `pending` | `accepted` | `rejected` | `withdrawn`). Response: { "bids": [...], "counts": {...}, "win_rate": 100, "pipeline": 2800 } (`pipeline` = sum of pending bid amounts).

`PUT /bids/{id}` — same body as create; only while pending and the job is still open.

`DELETE /bids/{id}` — withdraws a pending bid.

Emergencies

`GET /emergencies` — open requests in the provider's city for services they offer; each includes `my_price` (this provider's price for that service).

`POST /emergencies/{id}/accept` — first to accept wins; creates a confirmed booking immediately. 409 if another provider already claimed it.

Wallet & payouts

`GET /wallet` — **Query:** `bucket` (`all` | `available` | `escrow`). Balances, lifetime/monthly earnings, 6-month trend, ledger entries, and recent withdrawal requests.

```
{
  "wallet": { "id":1, "available_balance": 2700, "escrow_balance": 0 },
  "total_earned": 2700, "earned_this_month": 2700,
  "earnings_series": [ {"label":"Feb","value":0}, "..." ],
  "ledger": {
    "bucket": "all", "counts": {"all":3,"available":1,"escrow":2},
    "entries": [ { "id":3, "bucket":"available", "type":"...", "amount":2700, "description":"...", "booking_reference":"BK Z6POMA" },
    "pagination": {...}
  ],
  "min_withdrawal": 500,
  "withdrawal_requests": [ { "id":1, "reference":"WD-...", "amount":2700, "status":"pending", "payout_method":"jazzcash", "method":... } ]
}
```

`POST /payout-method` — Body: `payout_method` (`bank` | `jazzcash` | `easypaisa`, required), `payout_account_title`, `payout_account_number` (required), `payout_bank_name` (required only if `payout_method = bank`). Response: { "profile": {...} }

POST /withdrawals — Body: `amount` (required, $\geq$ `min_withdrawal`, $\leq$ current available balance). Requires a payout method already saved. Response 201: { "message": "...", "withdrawal": {...} }

POST /withdrawals/{id}/confirm-receipt — two-party confirmation: after an admin marks a bank-transfer withdrawal `awaiting_confirmation` (sent, pending the provider's acknowledgement), the provider confirms they actually received it, flipping it to `paid`. 422 if the withdrawal isn't currently `awaiting_confirmation`. Response: { "message": "...", "withdrawal": {...} }

Settlements (paying off cash-commission debt)

For cash-collected bookings, commission is never deducted at source — it posts as a negative `available_balance` instead (see `WalletService::chargeCashCommission()`). A provider carrying this debt past `settlement_grace_days` (7 days by default) gets auto-suspended from accepting new bookings until it's settled.

GET /settlements — how much is currently owed + past settlement submissions.

```
{
  "owed": 500,
  "is_suspended": false,
  "settlements": [
    {
      "id": 1, "reference": "STL-...", "method": "bank_transfer", "method_label": "Bank transfer",
      "amount": 500, "confirmed_amount": null, "status": "pending", "admin_notes": null,
      "confirmed_at": null, "created_at": "..."
    }
  ],
  "pagination": { "current_page": 1, "last_page": 1, "total": 1 }
}
```

`status` $\in$ `pending` | `confirmed` | `rejected`. Submitting a settlement doesn't touch the wallet or lift the suspension by itself — only an admin confirming the amount actually received does (which also auto-unsuspends the provider once `available_balance` $\geq$ 0 again).

POST /settlements — Body: `method` (`cash` | `bank_transfer`, required), `amount` (required, $\leq$ `owed`), `screenshot` (required if `method` is `bank_transfer`, image, multipart, max 8MB). 422 if nothing is currently owed. Response 201: { "message": "...", "settlement": {...} }

Portfolio

| Method | Path | Body | Notes | ---|---|---| GET | `/portfolio` | - | { "photos": [...] } || POST | `/portfolio` | multipart: `photos[]` (images, up to remaining slots under a 12-photo cap), `caption` (optional) | 201, { "photos": [...just the new ones...] } || DELETE | `/portfolio/{photoId}` | - | { "message": "Photo removed." } |

7. Resource reference

Quick field reference for nested objects that recur throughout the API.

ProviderProfileResource

```
{
  "id": 21, "user_id": 23, "name": "Test Pro", "avatar_url": null, "business_name": null,
  "display_name": "Test Pro", "bio": null, "experience_years": 5,
  "rating_avg": 5.0, "reviews_count": 1, "city": "Karachi", "address": null,
  "latitude": null, "longitude": null, "status": "approved", "rejection_reason": null,
  "has_payout_method": true,
  "services": [ {...ProviderServiceResource, when loaded...} ],
  "portfolio": [ {...ProviderPortfolioPhotoResource, when loaded...} ]
}
```

`status` $\in$ `draft` | `pending` | `approved` | `rejected`.

ServiceResource — { "id", "category_id", "category": {...or omitted}, "name", "slug", "description", "base_price", "duration_minutes", "is_active" }

BidResource — { "id", "job_post_id", "job_post": {...summary...}, "provider": {...ProviderProfileResource...}, "amount", "proposed_date", "proposed_time", "m

```
} . status ∈ pending | accepted | rejected | withdrawn .
```

ContractResource — {

```
"id", "reference", "title", "description", "address", "latitude", "longitude", "city", "preferred_start_date", "status", "quoted_total", "items"
} . status ∈ submitted | quoted | accepted | rejected | in_progress | completed | cancelled .
```

EmergencyRequestResource — {

```
"id", "reference", "status", "service", "consumer", "address", "latitude", "longitude", "city", "notes", "quoted_price", "quoted_at", "accepted_at"
} . latitude / longitude are nullable. status ∈ open | quoted | accepted | declined | matched | cancelled (an admin sets
quoted_price and moves it to quoted ; the consumer then accepts/declines). my_price only populated on the provider board
endpoint.
```

SubscriptionResource — {

```
"id", "reference", "status", "plan", "provider", "address", "city", "next_visit_date", "visits_used", "is_cancellable", "bookings", "cancelled_at"
} . status ∈ pending_assignment | active | cancelled | completed .
```

PaymentResource — {

```
"id", "reference", "gateway", "amount", "credit_applied", "commission_rate", "commission_amount", "provider_amount", "status", "screenshot_url"
} . status ∈ pending | escrow | released | refunded | failed . screenshot_url is only non-null for gateway=bank_transfer
payments — the consumer's own uploaded transfer proof, absolute URL ready to display/download. commission_rate /
commission_amount / provider_amount are only ever non-null once a payment is released (they're written by
WalletService::release() / chargeCashCommission() ), and only visible to the requesting user if they're an admin or a
provider — a consumer viewing their own booking's payment always gets null for these three, since the commission split is
between the platform and the provider, not the consumer's business. Commission is per-provider now (see
ProviderProfile.commission_rate , set by an admin at approval time), not a single platform-wide rate.
```

WithdrawalRequestResource — {

```
"id", "reference", "amount", "status", "payout_method", "method_label", "admin_notes", "screenshot_url", "processed_at", "created_at"
} . screenshot_url is the admin's proof-of-transfer for a bank -method payout once processed (null until then) — same absolute-
URL pattern as PaymentResource .
```

DisputeResource — {

```
"id", "reference", "opened_by_role", "reason", "status", "resolution", "resolution_note", "resolved_at", "created_at", "photos" } .
status ∈ open | resolved | dismissed (resolution is set by an admin). photos is [{"id", "url"}] , only present when loaded
(both POST .../dispute and GET /disputes/{id} load it).
```

8. What's intentionally out of scope here

- **Admin panel** — stays web-only; not part of this API.
- **Job seeker (careers/recruitment) flows** — the mobile app is for customers + professionals, not the internal hiring board. The web-only flow (app/Http/Controllers/JobSeeker/* , app/Http/Controllers/Admin/CareerApplicationController.php) covers: a profile with skills stored as a real JSON array (tag/pill UI, not a comma string), an explicit "use my saved resume vs. upload a different one" choice when applying, and a per-application audit trail (career_application_events — one row per submit/status-change/note/withdrawal, so admin review history is never overwritten). None of this has an API surface; add one under /api/job-seeker/* the same way the consumer/provider routes were mirrored if the app ever needs it.
- **Corporate/B2B accounts, referrals dashboard** — not wired into this API pass; can be added the same way (they already have web controllers to mirror) if the app needs them later.
- **Push notification delivery** — this API stores/reads in-app Notification rows (badge counts, notification center), but does not yet register Expo/FCM/APNs device tokens or send pushes. That's a separate small piece of work (a device_tokens table + a dispatch step in Notifier) worth doing once the app shell exists.
- **Real payment gateways** — JazzCash/Easypaisa drivers exist server-side but are disabled by default (config/payments.php); only the mock gateway is enabled out of the box, same as the web app.

9. Testing this yourself

```
curl -X POST https://your-domain/api/register \
-H "Accept: application/json" -H "Content-Type: application/json" \
-d '{"name":"Jane","email":"jane@example.com","phone":"03001234567","role":"consumer","password":"password123","password_confirm":"password123"}'

# then, using the returned token:
curl https://your-domain/api/me \
-H "Accept: application/json" -H "Authorization: Bearer <token>"
```

Every endpoint in this document was hit with real requests against a local instance during development (full booking → escrow → completion → wallet release cycle, jobs/bids, emergency accept, contract creation, dispute open, and token revocation on logout all verified working) — including the completion-required-photos flow, the cash/bank-transfer completion-payment flow, the in-progress visit-charge cancellation, dispute photo evidence, and CNIC normalization on onboarding, all added in this pass.